

CIM — Continuous Integrity Monitor

Implementation Plan, Task Breakdown and Test Schedule

Phase 1 (Oct–Dec 2026) · Phase 2 (Jan–Apr 2027)

Hamza Farooq (BSCS23057) — MCP-side security

Fasih (BSCS23160) — Skills-side security

Advisors: Dr. Adnan Noor Mian · Co-advisor: Mr. Sheryar

Department of Computer Science, IT University, Lahore

Session 2026–2027

Version 1.3 — companion to the revised (security-only) proposal

v1.1: repository rules; v1.2: environment + process layer

v1.3: compilation fixes (floats, verbatim, tables, list labels)

1 Purpose and How to Read This Document

This document converts the revised CIM proposal (security-only scope) into an executable plan: 36 serially numbered tasks across two phases, with owners, time windows, dependencies, deliverables, acceptance tests, a system test catalogue, demo checklists, a risk register, a progress tracker, binding repository working rules (Sections 4.1–4.4), the development-environment specification (Section 4.5), and the repository process-and-automation layer (Section 5: CI/CD pipelines, AI-assistance governance, the history audit log, the progress folder, and the AI-onboarding files).

Serial numbering rule. Task IDs **T-01** to **T-36** are global and permanent. IDs are never renumbered, never reused, and never reassigned. A dropped task keeps its ID with status **DROPPED** and a reason. This guarantees zero mixing between team members and phases.

Code	Meaning
H	Hamza Farooq — sole owner, MCP-side security
F	Fasih — sole owner, skills-side security
J	Joint/overlapping task (both members; a named <i>lead</i> prevents deadlock)
Task statuses	NS, IP, DONE, BLOCKED, DROPPED (tracker, Section 14)
Folder tags	[H] / [F] / [JOINT] in the repository map (Sections 4.1–4.2)
pytest markers	<i>unmarked</i> (unit) / sys (system test) / lab (needs lab VMs; never in CI) — Section 5.1
Weeks	Oct W1, Nov W2, ... = week number within the stated month (W1 = first full week)

Table 1: Legend.

Phase framing.

- **Phase 1 — the system can *SEE*** (Oct–Dec 2026): inventory, integrity plane, call capture, host behaviour collection, behavioural envelopes, drift detection. Ends with a **live working demo in mid-December**.
- **Phase 2 — the system can *REASON and ACT*** (Jan–Apr 2027): correlation, trust scores, response (quarantine / rollback / re-baseline), attestation, LLM narration, and full adversarial evaluation. Ends with the **final demo in late April**.

2 Phase Gates (Definition of Done)

2.1 December 2026 gate — working demo

Target date: week of 14 December 2026 (adjust to departmental calendar; dry run the week before; record a backup video).

- C1** `cim inventory` shows every bound MCP tool (with schema digests) and every installed skill (with content hashes), all recorded in the append-only store.
- C2** Live benign session against HexStrike AI through the proxy; every tool call captured and visible.
- C3** **Rug-pull detection live:** swapping to a backdoored server version fires a schema-drift alert with a human-readable diff.
- C4** **Trigger-payload detection live:** a trigger tool fires a behavioural alert that names the drifted feature *while the artefact hash is shown unchanged on screen*.
- C5** `cim status` shows inventory, events, alerts; chain verification passes.
- C6** Backup video recorded; demo script committed to the repo.

Stretch (cut first if time crunches): skills-side trigger detection beat (C4 on the skills side), proxy latency tuning, CLI polish.

Guardrail: if T-11 (host collector) slips, the December demo shows skills-side integrity detection only, and the skills behavioural envelope moves to January week 1. C1–C6 are never cut.

2.2 April 2027 gate — full system

Target date: last week of April 2027.

- A1** Correlation engine classifies all four cases correctly (trigger-activated compromise / malicious-update candidate / legitimate update / benign).
- A2** Response works end-to-end on both sides: quarantine blocks, verified rollback restores, legitimate update re-baselines with provenance.
- A3** Attestation: off-host verifier passes a healthy host and flags a tampered monitor; nonce replay rejected.
- A4** Trust scores (scanner + integrity + behaviour) visible on the review interface.
- A5** LLM narration of at least one alert, local model only.
- A6** Full evaluation tables vs. both baselines, generated from raw logs by script.
- A7** Final live demo passes two consecutive rehearsals; backup video recorded.

3 System Architecture and Build Order

4 Build and Setup Guide

4.1 Repository layout and package structure (created in T-01)

The project uses **one shared repository** (a monorepo), cloned by both members — never two separate repositories (rationale in Section 4.2). All Python code lives under a single package named `cim` in a `src`-layout. The `src`-layout is deliberate: the project depends on the official MCP Python

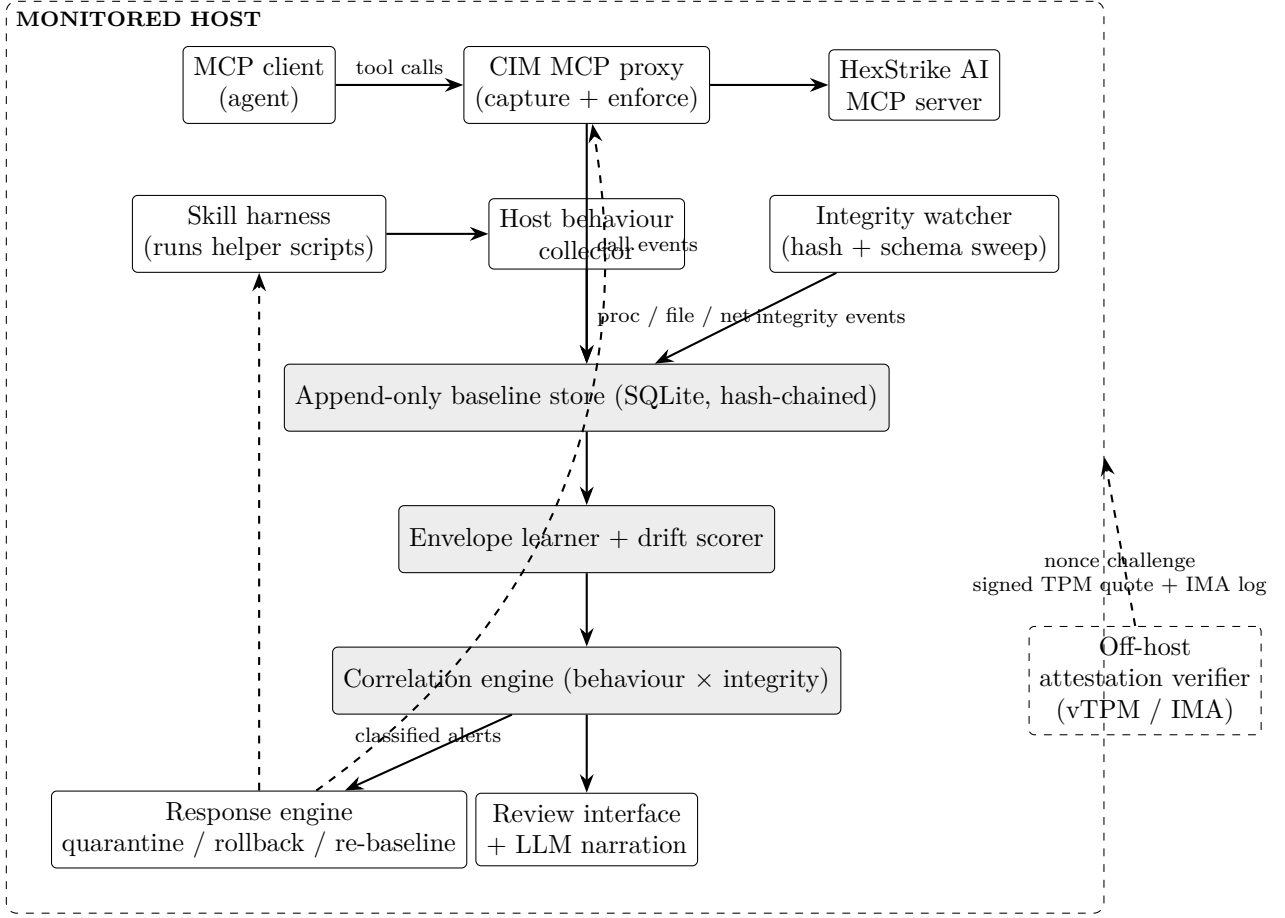


Figure 1: CIM architecture. Everything except the attestation verifier runs on the monitored host. Dashed arrows are enforcement and attestation paths.

Component	Tasks	Ready by
Repo, environment, CI/CD, process templates, schema contract, threat model	T-01–T-03	Oct W3
Lab environments (MCP + skills)	T-04, T-05	Oct W3
Baseline store	T-06	Oct W4
Artefact inventory (both sides)	T-07, T-08	Nov W1
Proxy collector / integrity watcher / host collector	T-09–T-11	Nov W4
Rug-pull detector	T-12	Nov W4
Integrated pipeline + CLI	T-13	Dec W1
Envelopes + drift scorer	T-14–T-16	Dec W2
Attack pack + December demo	T-17, T-18	Dec W3
Scanners, corpora, trust scores, correlation	T-19–T-24	Jan W4
Mid-review, attestation, baseline detectors	T-25–T-28	Feb W4
Response engines, update policy, LLM adjudication	T-29–T-32	Mar W4
Adversarial pass, dashboard, evaluation, final demo	T-33–T-36	Apr W4

Table 2: Build order at a glance.

SDK, whose import name is `mcp`; keeping our own code under `cim.*` (e.g. `cim.mcp.proxy`) guarantees that `import mcp` (the SDK) and our own `cim/mcp/` folder can never shadow each other.

The bracketed tags in the tree below give the owning member for every folder. They are **normative**: enforced by the rules of Section 4.2, mechanically by CODEOWNERS and CI, and socially by the review protocol of Section 4.4. Folders marked GITIGNORED exist on disk but are never committed.

```
cim/                                # ONE monorepo; both members clone it
  CONTEXT.md                        # [JOINT] AI/human onboarding; refreshed weekly
  DIRECTORY.md                     # [JOINT] GENERATED repo map; never hand-edited
  pyproject.toml                   # [JOINT] single package; pinned dependencies
  uv.lock                           # [JOINT] committed lockfile (exact versions)
  CODEOWNERS                       # [JOINT] auto-review routing
  .env.example                     # [JOINT] env-var template; real .env GITIGNORED
  .gitignore                       # [JOINT] .venv/ logs/ scratch/ .env VMs videos
  .pre-commit-config.yaml          # [JOINT] ruff + gitleaks hooks
  .github/workflows/               # [JOINT] ci-push.yml, merge-gate.yml (Sec 5.1)
  src/cim/                         # the single Python package "cim"
    common/                        # ---- shared border ----
      schema/                      # [F] owns; edits need BOTH + version bump
      store/                       # [H] implements, [F] reviews
      chain.py                     # [H] hash-chain verifier
      config.py                    # [JOINT]
    mcp/                           # [H] SOLE OWNER
    skills/                        # [F] SOLE OWNER
    correlation/                   # [H] leads T-24
    scanners/                      # mcp/ [H] , skills/ [F]
    attestation/                  # hostagent/ [H] , verifier/ [F]
    adjudicator/                  # [H] leads T-32
    dashboard/                     # [H] leads T-34
  scripts/                         # [JOINT] bootstrap, check-env, generators, demos
  tests/                           # mirrors the split
    mcp/ [H]   skills/ [F]         # unit tests per territory
    common/ [JOINT]                # contract round-trip + import smoke test
    sys/ [JOINT]                   # SYS-xx suite (markers: sys / lab)
  corpora/                         # mcp/ [H] , skills/ [F] , manifests [JOINT]
  history/                         # [JOINT] audit log (Sec 5.3)
    entries/                       # one entry per merged PR
    weekly/                        # weekly digests
  progress/                        # [JOINT] SINGLE SOURCE OF TRUTH (Sec 5.4)
    tasks/                         # T-XX.md, one file per task
    milestones/                    # dec-demo-2026.md, apr-demo-2027.md
    weekly/                        # sync notes: done / next / blocked
  docs/                           # [JOINT] threat model, runbooks, demo scripts
    decisions/                     # ADR-NNN decision records (Sec 5.8)
  logs/                            # [JOINT] runtime output -- GITIGNORED (Sec 5.7)
  scratch/                         # [JOINT] experiments -- GITIGNORED (Sec 5.6)
```

4.2 Repository ownership map and change-control rules

Both members work in the same repository; each owns a coherent territory. Git conflicts occur only when two people change the same lines of the same file, so this ownership map is drawn to make conflicts *structurally near-impossible* across territories (nothing in `mcp/` and `skills/` can conflict with each other) and to concentrate them on the small, deliberately shared border — where the joint review protocol of Section 4.4 applies.

Binding rules.

R1 One repository, never two. Both members clone the same monorepo. Two repos are explicitly forbidden — they hide schema drift until “integration week” and have no home for the shared drift-scorer library (T-16).

R2 Sole territories. The owner commits on a task branch; the other member reviews the PR. The

Area	Owner	Change-control rule
src/cim/mcp/, tests/mcp/, corpora/mcp/	H (sole)	H edits freely on a task branch; F reviews every PR. F does not push here without H's written agreement first.
src/cim/skills/, tests/skills/, corpora/skills/	F (sole)	Mirror rule: F edits, H reviews. H does not push here without F's written agreement first.
src/cim/common/schema/	F (T-02 lead)	Border. PR only; BOTH review; mandatory version bump; schema changes announced at the weekly sync <i>before</i> the PR is opened.
src/cim/common/store/, chain.py	H (T-06 impl)	Border. PR only; F reviews.
src/cim/common/config.py	J	Border. PR only; both review.
src/cim/correlation/	H leads (T-24)	PR; F reviews.
scanners/mcp/ / scanners/skills/	H / F	Each owns his wrapper; the other reviews.
attestation/hostagent/ / verifier/	H / F	Each owns his component; the other reviews.
adjudicator/, dashboard/	H leads (T-32, T-34)	PR; F reviews.
history/, progress/, docs/ (incl. decisions/), scripts/, CONTEXT.md	J	Border. PR only; both review. History entries are drafted by the PR author (AI-assisted per Sec. 5.2) and land with the PR.
DIRECTORY.md	J (generated)	Regenerated by <code>scripts/gen_directory.py</code> ; never hand-edited (rule R6); the merge gate fails any PR that changes the tree without regenerating it (Sec. 5.1).
tests/common/, tests/sys/	J	PR; both review (contract round-trip tests and the SYS-xx suite).
Root files: pyproject.toml, uv.lock, CI workflows, README, CODEOWNERS, .env.example, pre-commit config	J	PR only; one dependency change per PR; pinned versions; new dependencies announced at the weekly sync <i>before</i> being added.

Table 3: Territory map: who touches what, and under which rule.

non-owner may open a PR into the other's territory only after asking in writing (issue or PR comment) and getting agreement.

R3 Border files (`common/`, `history/`, `progress/`, `scripts/`, `docs/`, root files, `tests/sys/`) are PR-only, reviewed by both members. Schema changes additionally bump the contract version.

R4 main is protected. No direct pushes, ever. All changes arrive via reviewed PRs that pass the merge gate (Sec. 5.1).

R5 Full suite green before merge. Every PR runs both sides' tests in CI; a merge that breaks the other side's tests is forbidden.

R6 Generated files are regenerated, never hand-edited (`DIRECTORY.md`, the LaTeX tracker table via `gen_tracker.py`).

Why one monorepo and not two. (1) The project's top technical risk is the integration seam (T-02/T-13); one repo makes any schema drift visible the moment either member pulls, instead of at integration time. (2) The drift scorer (T-16) is one shared library used by both sides and needs one home. (3) One CI pipeline runs both sides' tests on every PR, so neither member can merge something that breaks the other. (4) The December demo needs the integrated pipeline (T-13); one repo makes integration continuous and boring instead of a scary December event. (5) Per-folder Git history gives the advisers verifiable evidence of who contributed what at defence time.

Enforcement. A CODEOWNERS file routes review requests automatically (replace the placeholder handles with the real GitHub usernames in T-01). GitHub uses the *last matching pattern*, so the catch-all both-members rule comes first and single-owner overrides follow it.

```
# CIM CODEOWNERS -- replace handles with real GitHub usernames.
# Rule: the LAST matching pattern wins, so the catch-all (both
# members) is first and single-owner overrides come after it.
*                                @hamza-user @fasih-user
# the contract: BOTH members review every schema change
/src/cim/common/schema/         @fasih-user @hamza-user
# sole territories
/src/cim/mcp/                    @hamza-user
/src/cim/skills/                 @fasih-user
/src/cim/scanners/mcp/          @hamza-user
/src/cim/scanners/skills/       @fasih-user
/src/cim/attestation/hostagent/ @hamza-user
/src/cim/attestation/verifier/  @fasih-user
# lead-owned components
/src/cim/correlation/           @hamza-user
/src/cim/adjudicator/           @hamza-user
/src/cim/dashboard/             @hamza-user
# tests and corpora mirror the split
/tests/mcp/                      @hamza-user
/tests/skills/                   @fasih-user
/corpora/mcp/                    @hamza-user
/corpora/skills/                 @fasih-user
```

4.3 Branching and review workflow

Branch naming. One branch per task, named `tXX-slug` where `XX` is the global task ID (e.g. `t09-mcp-proxy`, `t11-host-collector`, `t16-drift-scorer`). This keeps branches traceable to the task tracker — no mixing.

Commit and PR-title convention. Commit messages follow `T-XX(scope): summary` (e.g. `T-09(proxy): add frame logger`). PR titles follow `T-XX: description`; the merge gate enforces this pattern mechanically (Sec. 5.1), so no PR can merge without its task ID.

Workflow (every task, both members, no exceptions).

1. `git checkout main && git pull` — always branch off a fresh `main`.
2. `git checkout -b tXX-slug` — create the task branch.
3. Work *only* inside your territory for that task.
4. Run the *full* test suite (both sides) locally before pushing.
5. `git fetch && git rebase origin/main` — catch up with `main` before opening the PR.
6. Push and open a PR titled `T-XX: <short description>`; draft the history entry (Sec. 5.3) into the same PR so code and record land atomically.
7. The other member reviews (both members for border files, per Section 4.2); the GitHub-native AI reviewer (L1, Sec. 5.2) comments in parallel.
8. Merge only when CI is green and review is approved. Never push to `main` directly.

```
git checkout main && git pull          # branch off fresh main
git checkout -b t09-mcp-proxy         # branch name = task ID + slug
# ...work ONLY inside your territory...
pytest                               # FULL suite: both sides must pass
git fetch origin && git rebase origin/main
git push -u origin t09-mcp-proxy
# open PR titled "T-09: MCP proxy collector v1"
# other member reviews; then merge (no direct pushes to main)
```

Supporting rules. Branches are short-lived (days, not weeks — long branches are where conflicts breed). Pull `main` at the start of every working session. Identical pre-commit hooks (ruff formatting + gitleaks, from T-01) run on both machines, which silently eliminates the entire class of whitespace/import-order/quote-style conflicts *and* catches accidental secret commits locally.

4.4 Merge-conflict review protocol

With the territory map above, merge conflicts can only happen at the shared border. The hotspots, in order of likelihood:

Hotspot	Risk	Pre-agreed handling
<code>src/cim/common/schema/</code>	High	Never resolved alone; joint sitting; version bump; contract tests re-run (P2–P4 below).
<code>pyproject.toml</code> / <code>uv.lock</code>	Medium	One dependency change per PR keeps these rare; resolve jointly; re-pin versions.
Root files (CI, README, CODEOWNERS)	Low	Resolve jointly; verify CI on the resolved branch.
<code>history/</code> , <code>progress/</code> , <code>tests/sys/</code>	Low	Resolve jointly; re-run the affected tests.

Table 4: Conflict hotspots and pre-agreed handling.

Protocol (binding for both members).

- P1** A conflict inside a *sole-owner* folder is resolved by that owner; the other member still reviews the resolution as part of the PR.
- P2** A conflict touching any *border* file (`common/`, root files, `history/`, `progress/`, `tests/sys/`) is **never resolved alone**. Both members sit together — in person or on a call — and resolve it jointly, understanding what both sides intended before choosing anything.
- P3** Never blind-accept “ours” or “theirs” on a contract file (schema, store API). If either member cannot explain both sides of the conflict, the session stops until both can.
- P4** Schema conflicts additionally require a contract version bump and a green re-run of the round-trip tests in `tests/common/` before the PR can merge.

- P5** Every border conflict resolution is documented in the PR’s **history entry** (Sec. 5.3): what conflicted, what each side intended, how it was resolved and why, and who verified. This is written once, in the history entry — no duplicate record.
- P6** Never force-push to **main**; never rewrite history on a branch that has an open PR the other member is reviewing.
- P7** Two or more conflicts in the same file within a short period is a *process signal*, not a code problem: raise it at the weekly sync (the usual root cause is a border change that was not announced before the PR was opened).
- P8** If the members cannot agree on a resolution within one working day, the branch stays blocked and the decision escalates to the adviser. Neither member ever “wins by merging anyway.”

Practice run. T-01’s acceptance test includes a deliberate dry-run conflict: both members create conflicting edits to `common/config.py` on scratch branches, resolve them through this protocol, and record the resolution in a history entry — so the protocol is exercised once, in week 1, before it ever matters.

4.5 Development environment and toolchain

Three environments, one dependency truth. All dependencies come from one source: `pyproject.toml` with pinned versions plus the committed `uv.lock`.

- **E1 — dev machines** (both members): writing code, unit tests, lint, docs. Linux (Ubuntu 22.04/24.04) or **WSL2** on Windows — Docker, cgroups, inotify and auditd are Linux-only, so Windows members develop inside WSL2. macOS is acceptable for E1; the lab is always Linux.
- **E2 — lab VMs** (VM-MON, VM-TGT; Section 4.6): the monitored host and targets. Everything that touches the real monitored stack — the proxy against the pinned HexStrike build, the collectors, attestation — runs here.
- **E3 — CI runners** (GitHub-hosted): ephemeral, rebuilt from the lockfile on every run; approximates E1 (Section 5.1).

E1 toolchain (installed once per machine; every row verified by `scripts/check-env.sh`):

Tool	Min ver.	Purpose	Install	Verified by
Python uv	3.12 latest	Runtime for all components Virtualenvs + locked dependencies	<code>uv python install</code> Official installer	<code>check-env.sh</code> <code>uv -version</code>
Git	2.40	Version control (the monorepo)	Distro package	<code>git -version</code>
Docker Engine + CLI	24	Sandboxed scanning / re-baselining; vulnerable targets	Docker repository	<code>check-env.sh</code> (hello-world)
pre-commit	3.x	Local hooks: ruff + gitleaks	<code>uv tool install</code>	<code>pre-commit run</code>
Ollama	—	Local LLM adjudicator (<i>Phase 2, from March</i>)	ollama.com	<code>ollama list</code>
QEMU + swtpm	—	vTPM host (<i>February, E2 machine only</i>)	Distro packages	T-26 spike

Table 5: E1 toolchain. Phase-2-only tools (Ollama, QEMU/swtpm) are deliberately NOT part of the week-1 environment.

Initial dependency set (all pinned; grows only via the dependency-change rule of Section 4.2 — announced at the weekly sync *before* being added, one dependency per PR): runtime — the official

mcp SDK, pydantic, typer, watchfiles, psutil; dev group — pytest, pytest-cov, ruff, mypy, pre-commit.

Bootstrap. One command after cloning, and after any dependency change:

```
# scripts/bootstrap.sh -- run once after cloning, and after any
# dependency change (announced per the Section 4.2 rule)
uv venv .venv                # one venv per clone; gitignored
source .venv/bin/activate
uv sync                      # EXACT pinned versions from uv.lock
pre-commit install           # ruff + gitleaks on every commit
scripts/check-env.sh --dev   # verify everything works
```

Rules. Never `sudo pip`; never global installs; one `.venv/` per clone. The `cim` CLI and `cim.*` imports come from the editable install — there is no system-level install, ever.

Verification — `scripts/check-env.sh`. The single guarantee that everything is installed and works:

```
# scripts/check-env.sh --dev | --ci    (exit nonzero on any FAIL)
PASS python >= 3.12
PASS running inside a virtualenv (never global python)
PASS import cim, cim.mcp, cim.skills, ... (import smoke)
PASS cim --help
PASS docker daemon reachable (hello-world container)
PASS pre-commit hooks installed          [--dev only]
PASS git identity configured
FAIL? -> fix the environment BEFORE writing any code
# CI runs check-env.sh --ci on every push, so the three
# environments (dev machines, lab, CI) cannot silently drift.
```

Run it after cloning, after any environment change, and before every demo.

Docker in this project. Docker Engine + CLI on the dev machines and VM-MON; used for sandboxed re-baselining (Section 4.11), isolated scanner runs (Section 4.13) and the vulnerable targets on VM-TGT (Section 4.6) — consistent with the proposal’s isolation choice. After installation: add the user to the `docker` group (no `sudo docker`) and verify with `docker run -rm hello-world`.

Environment variables. A committed `.env.example` documents every variable the code reads; the real `.env` is gitignored and never committed; secrets are additionally caught by gitleaks in pre-commit and CI (Section 5.1).

4.6 Lab topology (T-04, T-05)

Hypervisor (KVM or VirtualBox) with a host-only network `10.10.10.0/24`; no route from lab VMs to the internet or any production network.

- **VM-MON** (Ubuntu 24.04): monitored host — pinned HexStrike AI (commit hash recorded in repo), skill corpus, all CIM components; vTPM attached from February (T-26).
- **VM-TGT**: deliberately vulnerable targets (vulhub containers or Metasploitable 2) for benign scanning workloads.
- **Attacker listener**: a lab-internal netcat/Python listener that stands in for exfiltration destinations in trigger payloads.
- Base + weekly snapshots of every VM; provisioning scripts in the repo so a clean rebuild takes under 30 minutes.

4.7 Event schema and baseline store (T-02, T-06)

One versioned JSON event schema is the contract between both sides:

```
{
  "schema": 1,
  "ts": "2026-11-03T14:22:31Z",
  "artefact": {"type": "mcp_tool", "server": "hexstrike",
               "tool": "nmap_scan"},
  "kind": "behaviour",
  "payload": {"value_classes": ["private_ip", "cidr"],
              "result_bytes": 8123, "ok": true}
}
```

Store tables (SQLite, WAL mode, append-only API only):

```
CREATE TABLE artefact(id INTEGER PRIMARY KEY, type TEXT,
                      name TEXT, origin TEXT);
CREATE TABLE record(id INTEGER PRIMARY KEY, artefact_id INTEGER,
                    kind TEXT,
                    payload TEXT,    -- canonical JSON
                    prev_hash TEXT, record_hash TEXT, ts TEXT);
-- record_hash = SHA256(prev_hash || kind || canonical_json(payload))
```

Tamper evidence: `cim verify-chain` recomputes the chain; any edit or deletion of a historical row breaks it.

4.8 MCP proxy (T-09, T-12, T-29)

- The client launches the *proxy* instead of the server; the proxy spawns the real HexStrike process as a child and pipes stdio.
- MCP stdio transport is newline-delimited JSON-RPC: read a line, parse, log, forward, untouched. Capture `initialize`, `tools/list` (schema digests feed T-12) and `tools/call` (name, arguments, result digest + size, timestamp, order).
- v1 fails open (parse failure → passthrough + alert); this availability trade-off is documented (ADR) and revisited in the adversarial pass (T-33).
- Enforcement hook (Phase 2): quarantined tools are refused at the proxy and removed from the advertised tool list — no agent-side change.

4.9 Skill harness and host collector (T-05, T-11)

- The harness (not an LLM agent) loads `SKILL.md`, lists helper scripts, and executes them as subprocesses with a clean environment and scripted representative arguments — deterministic and reproducible.
- Each invocation gets its own **cgroup v2** named with the skill ID; processes, file writes and sockets are attributed via cgroup membership (survives double-forked grandchildren). `auditd` + `psutil` are the fallback attribution path.
- Egress connections observed by polling `/proc` per cgroup.

4.10 Envelopes and drift scoring (T-14, T-15, T-16)

- Per-tool features: call rate, argument type/shape signature, value classes (private vs public IP, domain vs IP, path roots), result size, error rate. Per-skill features: binaries executed, file-write targets, network destinations, event rates per invocation.
- Rolling window (default: last 100 observations). Numeric features get z-scores; categorical features get a support set with a *never-seen-value* flag. Composite score = max deviation; alert carries the top-3 drifted features (the explainability story).
- Warm-up gate (default 30 observations) before any scoring; thresholds calibrated on the benign corpus.

4.11 Correlation and response (T-24, T-29, T-30)

State machine joining artefact integrity state $\times$ behaviour state: unchanged + drifted $\rightarrow$ *trigger-activated compromise*; changed + drifted $\rightarrow$ *malicious-update candidate*; changed + in-envelope + valid provenance $\rightarrow$ *legitimate update* (re-baseline); ambiguous $\rightarrow$ review queue, never silently dropped. Responses: quarantine (proxy refusal / harness refusal + quarantine directory), verified rollback (restored digest must equal baseline), re-baseline (append, never overwrite).

4.12 Attestation (T-26, T-27)

QEMU VM with `swtpm` (vTPM); kernel booted with `ima_policy=tcb` so every executed file (including CIM's own binaries) extends PCR 10. Off-host verifier issues a fresh nonce; the host agent returns a signed TPM quote plus the IMA measurement log; the verifier checks the quote signature and replays the log against the PCR, then compares CIM digests against expected values. **Early spike (Oct/Nov, one day)**: boot a throwaway VM with `swtpm` + IMA and observe one measurement, so February is not first contact. **Fallback if vTPM plumbing overruns**: IMA log monitoring + signed manifest verification, scope-reduced and documented (ADR).

4.13 Scanner integration and trust scores (T-19, T-20, T-23)

Thin wrappers around Cisco mcp-scanner and NVIDIA SkillSpector, pinned versions, findings mapped onto artefact IDs, failures degrade gracefully. Trust score combines static risk, integrity state and behavioural score with configured weights; composition is monotonic (no worsening component can raise a score); the formula is documented in T-23. **Early smoke test (this month, 2 hours)**: run both scanners once in the lab so January integration holds no surprises.

4.14 Corpora and evaluation protocol (T-21, T-22, T-35)

Benign / mutation / trigger corpora per side, generated by scripts, checksummed and versioned. Matched-FPR methodology: run CIM and both baseline detectors on the identical corpora; choose thresholds so false-positive rates on the legitimate-update corpus are equal; then compare detection rates. All result tables and plots generated from raw logs by a single command.

4.15 Ethics and safety note

All offensive tooling runs only against the team's own isolated VMs. No real credentials anywhere in the corpus (dummy tokens only); all exfiltration destinations are lab-internal listeners. This note appears in the thesis and the demo preamble.

5 Repository Processes and Automation

This section specifies the process layer delivered with T-01 and used for the rest of the project: the CI/CD pipelines, the governance rules for AI assistance, the history audit log, the progress folder (the single source of truth for status), the AI-onboarding files, and the scripts/scratch split. One design principle runs through all of it: **GitHub stores raw events; the repository stores interpretation; progress/ is the single source of truth for status**. Meta-work is capped at roughly 10% of weekly hours: week 1–2 builds only the essentials below; further automation is added only when manual cost is proven (risk R10).

5.1 CI/CD pipelines (T-01)

Two workflows live in `.github/workflows/`.

Push pipeline (`ci-push.yml`) — runs on every push to every branch; target wall time under five minutes:

1. Python 3.12 + `uv sync` from the lockfile (cached between runs).
2. `ruff format -check + ruff check` (lint and formatting).
3. `mypy src/cim` (types; catches unimplemented functions and signature drift).
4. **Import smoke test**: `tests/common/test_imports.py` imports every module in the `cim` package — the fastest possible “missing file / broken syntax / missing code” detector; fails in seconds.
5. Secrets scan (gitleaks).
6. Unit suite (`pytest -m "not sys and not lab"`) with a **coverage gate** — starts at 60%, ratcheted monthly; coverage is the detector for “missing code that imports fine”.
7. Scripts smoke: every file in `scripts/` must run `-help` successfully.
8. `scripts/check-env.sh -ci` (Section 4.5), so dev machines and CI cannot drift.

Merge gate (`merge-gate.yml`) — runs on every PR targeting `main`; contains everything the push pipeline has, plus:

9. The full unit suite plus the contract round-trip tests (`tests/common/`).
10. CI-runnable system tests (`pytest -m "sys and not lab"`): `SYS` tests that spawn local fixture MCP servers (HexStrike is Python and runs in CI containers). `lab`-marked tests need the lab VMs and never run in CI.
11. **PR-title linter**: the title must match the pattern `^T-\d{2}: .+` — no PR merges without its task ID, mechanically enforcing the traceability rule.
12. **DIRECTORY.md freshness**: CI regenerates the repo map; a PR that changes the tree without regenerating `DIRECTORY.md` fails.

Branch protection on main (repository settings): PRs only, no direct pushes; exactly one approving review required (the other member); the merge-gate checks are required status checks; force-push disabled; CODEOWNERS review required on border paths (Section 4.2).

Test markers (documented so every run is reproducible): `unmarked` = unit; `sys` = system test; `lab` = needs lab VMs (excluded from all CI); `slow` = long-running. Push CI runs `-m "not sys and not lab"`; the merge gate runs `-m "not lab"`; full `pytest` (including `lab`) runs in the lab before each demo.

5.2 AI assistance governance

AI is used in three distinct layers with distinct jobs:

- **L1 — AI reviewer (GitHub-native, automatic)**. An AI code reviewer (GitHub Copilot code review or the Claude GitHub Action) is enabled on the repository and posts line-level review comments on every PR. These live on GitHub, attached to the diff, permanently. L1 findings are *advisory*: the reviewer of record is always the other human member; how each L1 finding was addressed (or why it was dismissed) is recorded in the history entry.
- **L2 — AI chronicler (into history/, author-verified)**. While a PR is open, the author gives the AI the diff and the review discussion; the AI drafts the history entry from the Section 5.3 template; the author verifies it and commits it as part of the PR.
- **L3 — AI analyst (into progress/, both-verified)**. At each weekly sync and milestone, the AI drafts the status / milestone report from the tracker, the merged PRs and the history entries; both members verify before commit.

Binding rules.

- G1** The AI never commits, pushes or merges by itself. No automation with write access. If automation is added later, the AI may open PRs but never merge them.
- G2** Every AI-drafted artifact in the repository is attributed and dated (“AI-drafted, verified by H/F”) and human-verified before commit.

G3 No secrets, credentials or engagement data in AI prompts — dummy tokens only, the same rule as the corpora (Section 4.15).

G4 AI-suggested code goes through the same PR review as human code; tests land in the same PR as the code they test.

This governance is deliberately conservative: it is also the project’s answer to academic-integrity questions at the defence — *AI-drafted, human-verified, attributed, dated*.

5.3 History folder (audit log)

GitHub already stores every push, PR, comment and review permanently. `history/` stores what GitHub cannot: the *interpretation* — what each change meant, what the AI review found, and how conflicts were resolved and why (including whose code was preferred in a conflict, and the reason).

```
history/
  entries/PR-0042_T-09.md    # one file per merged PR
  weekly/2026-W43.md        # weekly digest (drafted at the sync)
```

Entry template — fixed fields:

```
# PR-0042 - T-09: MCP proxy collector v1
- PR: #42 | Commit: <merge sha> | Merged: 2026-11-20
- Author: H | Reviewer: F | Territory: mcp
- Summary: one paragraph, what changed and why
- AI review (L1): clean | <findings and how each was addressed>
- Tests added: <list>
- Conflict record: NONE | conflicted files; each side’s intent;
  resolution (WHOSE code was preferred and WHY); protocol rule
  applied (P1-P8); verified by both members
- Notes: deviations, follow-up issues
```

Rules.

H1 One entry per merged PR, drafted (L2) and committed *as part of the PR itself*, so code and record land atomically.

H2 The conflict record is protocol P5’s resolution note (Section 4.4) — written once, here; no duplicate.

H3 Entries record the PR number and commit SHA, not just the URL.

H4 The weekly digest is drafted at the sync (L3): one line per merged PR plus status vs. plan.

H5 Merged entries are never rewritten; corrections go into a later note.

5.4 Progress folder (single source of truth)

Status must live in exactly one place. The division of truth:

Artifact	Role
Chat memory card	conversation continuity between AI sessions
CONTEXT.md	repo orientation for any human or AI
progress/	canonical task and milestone status
history/	audit log of merged changes and conflict resolutions
GitHub	raw permanent record of all pushes, PRs, comments

Table 6: Division of truth. `progress/` is the single source of truth for status.

```

progress/
  tasks/T-09.md          # canonical status, one file per task
  milestones/dec-demo-2026.md # C1-C6 checklist + evidence
  milestones/apr-demo-2027.md # A1-A7 checklist + evidence
  weekly/2026-W43.md     # sync notes: done / next / blocked

```

Task-file template — fixed fields:

```

# T-09 - MCP proxy collector v1
- Owner: H | Status: DONE | Window: Nov W1-W3 | Completed: 2026-11-22
- Acceptance tests:
  - [x] 100/100 scripted replay logged (evidence: PR #42, SYS-03)
  - [x] malformed frames do not crash (evidence: PR #42)
  - [ ] p95 latency <= 50 ms (evidence: pending)
- Evidence: PR #42
- Deviations: window slipped 3 days (midterms); no impact on T-14

```

Milestone files mirror the C1–C6 / A1–A7 checklists of Section 2, each marked achieved / not achieved with links to evidence (PRs, run outputs, video). `scripts/gen_tracker.py` renders the LaTeX tracker table (Section 14) directly from `progress/tasks/*.md`; run it before regenerating this PDF. **No hand-maintained status tables anywhere.** Progress files are AI-drafted (L3) at the sync, human-verified, and committed like any code.

5.5 CONTEXT.md and DIRECTORY.md (AI onboarding)

CONTEXT.md (repo root; at most one page; refreshed at every weekly sync) is the onboarding file for any AI assistant or new human. Ordered contents: (1) what CIM is, two lines, plus the demo host and pinned commit; (2) current phase, next milestone and its date; (3) active tasks right now, with owners; (4) the territory map in one line, pointing to **DIRECTORY.md** for the full map and to this document for the rules; (5) the binding rules R1–R6, one line each; (6) PR titles (T-XX: description), branches (tXX-slug), and “the AI never commits”; (7) the division of truth (Section 5.4); (8) how AI is used (L1/L2/L3, one line each, per Section 5.2).

The onboarding pack for any AI is: **CONTEXT.md** + **DIRECTORY.md** + this plan. Suggested first instruction to paste into any AI session:

```

"You are assisting with the CIM project. Read CONTEXT.md and
DIRECTORY.md first, then the plan document. Follow the territory
map and the PR rules; never commit or push; attribute every
draft; stop and ask when a rule is unclear."

```

DIRECTORY.md (repo root) is the generated repo map — one line per top-level folder — produced by `scripts/gen_directory.py`, never hand-edited (rule R6); the CI merge-gate freshness check (Section 5.1) keeps it current forever.

5.6 scripts/ and scratch/

`scripts/` holds *operational utilities humans run*: `bootstrap.sh`, `check-env.sh`, `gen_directory.py`, `gen_tracker.py`, `corpus-gen.py` (January), `vm-rebuild.sh`, `demo-run.sh`. AI may draft them (rule G4); each must support `-help`; CI smoke-checks that (Section 5.1).

`scratch/` holds experiments and is **gitignored**. **Promotion rule:** when a scratch script proves genuinely reusable, clean it up, add a docstring, and move it to `scripts/` via a PR (e.g. titled T-09(tools): promote log-parser from scratch). Reuse without a committed junk drawer. Never stage with `git add -A`; the `.gitignore` (`logs/`, `scratch/`, `.venv/`, `.env`, VM images, videos) is the backstop.

5.7 logs/

`logs/` holds *runtime output only* and is gitignored. Collectors emit structured JSON with rotation from day one, so demo and evaluation evidence is machine-parseable. Curated excerpts cited as evidence live in `progress/milestones/`; raw logs stay local.

5.8 Decision records (docs/decisions/)

One page per significant technical decision, named `ADR-NNN-title.md`: context, options considered, decision, consequences (e.g. ADR-001: proxy fails open in v1). Written when the decision is made, cited in the thesis; at the defence, “why did you choose X?” has a dated document instead of a shrug.

6 Phase 1 Task Catalogue (October – December 2026)

Table 7: Phase 1 summary (details below).

ID	Task	Owner	Window	Depends on
T-01	Project infrastructure, environment, CI/CD & working agreement	J (lead H)	Oct W1	–
T-02	Event & data contract v1.0	J (lead F)	Oct W1–W2	T-01
T-03	Threat model v1.0	J	Oct W2–W3	T-01
T-04	MCP lab environment (HexStrike pinned)	H	Oct W1–W3	T-01
T-05	Skills lab environment + harness + seed corpus	F	Oct W1–W3	T-01
T-06	Baseline store (append-only, hash-chained)	J (impl H)	Oct W3–W4	T-02
T-07	MCP artefact inventory + schema digests	H	Oct W4–Nov W1	T-04, T-06
T-08	Skills artefact inventory + hashing	F	Oct W4–Nov W1	T-05, T-06
T-09	MCP proxy collector v1 (critical path)	H	Nov W1–W3	T-02, T-04, T-06
T-10	Skills integrity watcher + sweep	F	Nov W1–W2	T-08
T-11	Host behaviour collector v1	F	Nov W2–W4	T-02, T-05, T-06
T-12	MCP schema-drift (rug-pull) detector	H	Nov W3–W4	T-07, T-09
T-13	Pipeline integration + <code>cim</code> CLI v0	J (lead F)	Nov W4–Dec W1	T-07–T-12
T-14	MCP behavioural envelope v1	H	Dec W1–W2	T-09, T-13
T-15	Skills behavioural envelope v1	F	Dec W1–W2	T-11, T-13
T-16	Drift scorer v1 (shared library)	J (lead H)	Dec W2	T-14, T-15
T-17	Attack pack v1 (4 canonical attacks)	J (split by side)	Dec W2–W3	T-12, T-16
T-18	December demo + backup video	J (H drives, F skills beat)	Dec W3	T-13, T-17

October 2026

T-01 — Project infrastructure, environment, CI/CD and working agreement

Owner: **J** (lead H) **Window:** Oct W1 **Depends on:** –

Deliverable: Monorepo per Section 4.1 (src-layout, package `cim`, gitignored `logs/` and `scratch/`, `history/`, `progress/`, `scripts/`, `CONTEXT.md`, `DIRECTORY.md`, `.env.example`, committed `uv.lock`); ownership map + `CODEOWNERS` with real usernames (Section 4.2); protected `main` with branch protection (Section 4.3); CI push pipeline + merge gate live per Section 5.1 (PR-title linter, `DIRECTORY` freshness check, coverage gate, secrets

scan); `scripts/bootstrap.sh` + `scripts/check-env.sh` passing on both machines and in CI (Section 4.5); templates committed for history entries, progress task files and weekly notes (Sections 5.3–5.4); ADR-001 (tooling decisions); Python 3.12 + uv, ruff/mypy/pytest/pre-commit on both machines; one issue per task with the task ID in the title; weekly 30-minute sync fixed; the merge-conflict protocol of Section 4.4 and the AI-governance rules of Section 5.2 adopted in writing by both members.

Acceptance test: Push CI green on a dummy test on a feature branch, and the merge gate green on its PR; `check-env.sh -dev` passes on both machines; the gates are proven to *fail when they should* — a PR with a lint error fails push CI, and a PR with a non-conforming title or a stale `DIRECTORY.md` is rejected by the merge gate; both members build and run the full suite from a clean clone in under 10 minutes; the deliberate dry-run conflict on `common/config.py` is resolved via Section 4.4 and recorded in a history entry.

T-02 — Event and data contract v1.0

Owner: J (lead F) **Window:** Oct W1–W2 **Depends on:** T-01

Deliverable: Versioned JSON schema for artefact records, integrity events, behaviour events and alerts; canonical-JSON hashing rule; store DDL. Both collectors will write exactly this schema. Governed by the border-change rules of Section 4.2 (PR only, both review, version bump).

Acceptance test: Both members round-trip sample events through the schema validator; schema version field present; signed off by both.

T-03 — Threat model v1.0

Owner: J **Window:** Oct W2–W3 **Depends on:** T-01

Deliverable: Document: assets, trust boundaries, attack matrix — A1 MCP rug pull, A2 MCP trigger tool, A3 skills helper exfil, A4 legitimate update (false-positive path), A5 monitor tampering (maps to attestation) — each mapped to the CIM phase that detects it.

Acceptance test: Every detection task (T-09 onward) traces to at least one attack row; adviser sign-off.

T-04 — MCP lab environment

Owner: H **Window:** Oct W1–W3 **Depends on:** T-01

Deliverable: VM-MON running HexStrike AI at a pinned commit (hash recorded); Python MCP client harness with scripted call workloads; VM-TGT vulnerable targets; host-only isolated network; snapshots; provisioning scripts in repo. Includes a one-day stdio-framing spike for T-09.

Acceptance test: Harness enumerates 150+ tools; VM-TGT unreachable from outside the lab subnet; clean rebuild under 30 minutes; spike logs a full client session through a passthrough pipe.

T-05 — Skills lab environment

Owner: F **Window:** Oct W1–W3 **Depends on:** T-01

Deliverable: Minimal skill runner: loads `SKILL.md`, executes helper scripts as subprocesses in a clean env, one cgroup v2 per invocation; seed corpus of 10+ benign and 3+ suspicious skills, checksummed into the repo.

Acceptance test: Runner executes every corpus skill; cgroup membership observable per invocation; corpus manifest verifies.

T-06 — Baseline store

Owner: J (design both, implementation H, review F) **Window:** Oct W3–W4 **Depends on:** T-02

Deliverable: SQLite WAL store with append-only API, hash-chained records, `verify-chain` CLI (schema in Section 4.7).

Acceptance test: Unit tests: no update/delete code path exists; tampering with any historical row breaks `verify-chain`; 10,000 events appended in under 5 seconds.

November 2026

T-07 — MCP artefact inventory

Owner: **H** **Window:** Oct W4–Nov W1 **Depends on:** T-04, T-06

Deliverable: Enumerate configured MCP servers; fetch `tools/list`; canonical-JSON digest per tool schema + server manifest; baseline records written to the store.

Acceptance test: Digests stable across re-reads; changing one tool description changes exactly one digest; full HexStrike inventory stored.

T-08 — Skills artefact inventory

Owner: **F** **Window:** Oct W4–Nov W1 **Depends on:** T-05, T-06

Deliverable: Enumerate skill roots; parse `SKILL.md` metadata; SHA-256 of `SKILL.md` and every helper script; baselines written.

Acceptance test: N installed skills produce N baselines; a one-byte edit flags exactly that artefact; a newly added helper file is caught on sweep.

T-09 — MCP proxy collector v1 (critical path)

Owner: **H** **Window:** Nov W1–W3 **Depends on:** T-02, T-04, T-06

Deliverable: Transparent stdio proxy per Section 4.8: parses newline-delimited JSON-RPC; logs initialize / tools list / tools call (name, args, result digest + size, timestamp, order) into the store; config via environment; enforcement hook stubbed for Phase 2.

Acceptance test: 100-call scripted replay logged 100/100 with correct ordering; malformed frames do not crash the proxy; p95 added latency ≤ 50 ms; a real client session completes normally through the proxy.

T-10 — Skills integrity watcher

Owner: **F** **Window:** Nov W1–W2 **Depends on:** T-08

Deliverable: inotify watcher on skill roots plus a periodic sweep (default 60 s); integrity events (create / modify / delete / rename) into the store.

Acceptance test: File edit produces an event within 5 s via inotify and within the sweep interval when inotify is disabled; read-only access produces no event; rename-and-replace caught.

T-11 — Host behaviour collector v1

Owner: **F** **Window:** Nov W2–W4 **Depends on:** T-02, T-05, T-06

Deliverable: Per-invocation observation of helper execution: process tree (psutil), file writes, egress connections, attributed via cgroup membership (auditd fallback); behaviour events into the store (Section 4.9).

Acceptance test: A helper that spawns a subprocess, writes a file and opens a socket yields all three events, attributed to the correct skill, including a double-forked grandchild; an idle benign skill yields zero events.

T-12 — MCP schema-drift (rug-pull) detector

Owner: **H** **Window:** Nov W3–W4 **Depends on:** T-07, T-09

Deliverable: Re-fetch `tools/list` on reconnect and on sweep; diff against pinned baseline; schema-drift event with a human-readable diff (added parameters, changed descriptions).

Acceptance test: A postmark-style replica (clean v1.0 $\rightarrow$ backdoored v1.1) fires a drift event naming the changed tool and parameter; a benign re-read fires nothing.

December 2026

T-13 — Pipeline integration and CLI v0

Owner: **J** (lead F) **Window:** Nov W4–Dec W1 **Depends on:** T-07, T-08, T-09, T-10, T-11, T-12

Deliverable: Single cim CLI: `inventory / watch / status / events / verify-chain`; one config file; both collectors run as supervised processes with a watchdog. **This is the joint integration point.**

Acceptance test: Clean-machine bootstrap; 10 scripted benign operations produce a correct inventory and zero alerts; one tampered artefact produces exactly one alert; a killed collector is restarted by the watchdog.

T-14 — MCP behavioural envelope v1

Owner: **H** **Window:** Dec W1–W2 **Depends on:** T-09, T-13

Deliverable: Per-tool feature extractor (Section 4.10): call rate, argument shape signature, value classes, result size, error rate; rolling window with warm-up counter.

Acceptance test: A 200-call benign replay populates the envelope deterministically (same input → identical features); warm-up state exposed via CLI.

T-15 — Skills behavioural envelope v1

Owner: **F** **Window:** Dec W1–W2 **Depends on:** T-11, T-13

Deliverable: Per-skill profile: binaries executed, file-write targets, network destinations, event rates per invocation; rolling window with warm-up.

Acceptance test: Scripted benign corpus yields stable, reproducible profiles across three consecutive runs.

T-16 — Drift scorer v1 (shared library)

Owner: **J** (lead H) **Window:** Dec W2 **Depends on:** T-14, T-15

Deliverable: Per-feature deviation (rolling z-score + never-seen-value flag), composite score, configurable threshold, warm-up gating; alerts carry top-3 drifted features. **Shared library used by both sides — lives at the border, change-controlled per Section 4.2.**

Acceptance test: Post-warm-up benign replay of 100+ executions yields zero alerts; an injected never-seen argument class fires an alert naming that feature; the threshold sweep is recorded in the repo.

T-17 — Attack pack v1

Owner: **J** (H builds MCP attacks, F builds skills attacks) **Window:** Dec W2–W3 **Depends on:** T-12, T-16

Deliverable: Four scripted attacks: A1 rug-pull replica; A2 MCP trigger tool (conditional exfiltration); A3 skills trigger payload (keyword → env exfiltration); A4 legitimate update (false-positive control); each mapped to its expected detector.

Acceptance test: Each attack fires reliably; A1 detected by T-12; A2 and A3 detected by T-16 *while artefact hashes remain unchanged (verified in the log)*; A4 produces no behavioural alert.

T-18 — December demo

Owner: **J** (H drives the demo, F runs the skills beat) **Window:** Dec W3 (dry run end of Dec W2) **Depends on:** T-13, T-17

Deliverable: 10-minute scripted live demo per the Section 12 checklist, against HexStrike + the skills host; demo script and runbook committed; backup video recorded.

Acceptance test: Two consecutive full rehearsals pass; all core beats C1–C5 present; backup video exists.

7 Phase 2 Task Catalogue (January – April 2027)

Table 8: Phase 2 summary (details below).

ID	Task	Owner	Window	Depends on
T-19	mcp-scanner integration + static risk	H	Jan W1–W2	T-07
T-20	SkillSpector integration + static risk	F	Jan W1–W2	T-08

ID	Task	Owner	Window	Depends on
T-21	MCP benchmark corpora	H	Jan W2–W4	T-09, T-14
T-22	Skills benchmark corpora	F	Jan W2–W4	T-11, T-15
T-23	Trust score + shared-inventory wiring	J (lead F)	Jan W3–W4	T-16, T-19, T-20
T-24	Correlation engine	J (lead H)	Jan W2–W4	T-10, T-12, T-16
T-25	Mid-project review package	J (lead F)	Feb W1	T-18, T-24
T-26	Attestation: measured host (vTPM + IMA)	H	Feb W1–W3	T-13
T-27	Attestation: off-host verifier	F	Feb W2–W4	T-26
T-28	Baseline comparison detectors	J (lead F)	Feb W2–W4	T-19–T-22
T-29	Response engine, MCP side	H	Mar W1–W2	T-09, T-23, T-24
T-30	Response engine, skills side	F	Mar W1–W2	T-10, T-23, T-24
T-31	Update policy + false-positive bounding	F	Mar W2–W3	T-22, T-30
T-32	LLM adjudication via Ollama	J (lead H)	Mar W2–W4	T-23, T-24
T-33	Adversarial hardening pass	J (F attacks, H hardens)	Mar W4–Apr W1	T-26, T-27, T-29, T-30
T-34	Thin review interface + usability check	J (lead H)	Apr W1–W2	T-23, T-27, T-29, T-30
T-35	Full evaluation run	J (lead F)	Apr W2–W3	T-21, T-22, T-28, T-29, T-30
T-36	Final demo + defence preparation	J (both)	Apr W4 (into May)	T-33, T-34, T-35

January 2027

T-19 — mcp-scanner integration

Owner: **H** **Window:** Jan W1–W2 **Depends on:** T-07

Deliverable: Wrapper around Cisco mcp-scanner (pinned version): runs at bind time and on demand; findings parsed into per-tool static-risk records in the store.

Acceptance test: Findings mapped to correct tool IDs; re-scan idempotent; a known-suspicious test server is flagged; scanner failure degrades to an alert, not a crash.

T-20 — SkillSpector integration

Owner: **F** **Window:** Jan W1–W2 **Depends on:** T-08

Deliverable: Wrapper around NVIDIA SkillSpector: install-time and on-demand scans; per-skill static-risk records.

Acceptance test: Suspicious seed corpus flagged; benign corpus below threshold; graceful degradation on scanner failure.

T-21 — MCP benchmark corpora

Owner: **H** **Window:** Jan W2–W4 **Depends on:** T-09, T-14

Deliverable: Benign corpus: 1000+ calls across 10+ tools (scripted generators); mutation corpus: 3+ rug-pull variants; trigger corpus: 3+ trigger types; all versioned, checksummed, reproducible.

Acceptance test: Regeneration produces identical event counts; corpus manifest verifies.

T-22 — Skills benchmark corpora

Owner: **F** **Window:** Jan W2–W4 **Depends on:** T-11, T-15

Deliverable: Benign: 300+ invocations; mutation: 3+ variants; trigger: 3+ types; reproducible generators.

Acceptance test: Same criteria as T-21.

T-23 — Trust score and shared-inventory wiring

Owner: J (lead F) **Window:** Jan W3–W4 **Depends on:** T-16, T-19, T-20

Deliverable: Documented monotonic formula combining static risk, integrity state and behavioural score into a per-artefact trust score; scores surfaced in `cim status` and the store with full history.

Acceptance test: Unit tests for composition and monotonicity (no worsening component can raise a score); score history retained per artefact.

T-24 — Correlation engine

Owner: J (lead H) **Window:** Jan W2–W4 **Depends on:** T-10, T-12, T-16

Deliverable: Per-artefact state machine joining behavioural drift with artefact-integrity state into the four classes of Section 4.11, plus an explicit review state; classified alerts emitted. **This is the project's core novelty.**

Acceptance test: Synthetic event sequences for all four classes classify correctly; the ambiguous case lands in the review state, never silently dropped.

February 2027

T-25 — Mid-project review

Owner: J (lead F) **Window:** Feb W1 **Depends on:** T-18, T-24

Deliverable: Slide deck + live walkthrough for the advisers (December demo + correlation progress); feedback log with owners and dates.

Acceptance test: Rehearsal completes within 20 minutes; every adviser comment logged with a follow-up task.

T-26 — Attestation: measured host

Owner: H **Window:** Feb W1–W3 **Depends on:** T-13

Deliverable: Monitored VM rebuilt with vTPM (swtpm/QEMU) and kernel IMA (`ima_policy=tcb`); CIM binaries and collectors automatically measured; IMA measurement-log capture service; VM snapshot saved.

Acceptance test: IMA log contains CIM binary hashes at execution; PCR 10 extends on each measurement.

T-27 — Attestation: off-host verifier

Owner: F **Window:** Feb W2–W4 **Depends on:** T-26

Deliverable: Verifier service: fresh nonce challenge → signed TPM quote → signature check → IMA log replay against PCR → verdict to CLI/dashboard; alert on mismatch (Section 4.12).

Acceptance test: Healthy host passes; stopping the monitor and patching a binary produces a fail verdict; a replayed old quote (nonce reuse) is rejected.

T-28 — Baseline comparison detectors

Owner: J (lead F) **Window:** Feb W2–W4 **Depends on:** T-19, T-20, T-21, T-22

Deliverable: (a) Install-time-scan-only detector and (b) hash-pinning-only detector, both runnable over the identical corpora with output in the same format as CIM.

Acceptance test: Both produce verdicts over the full corpora; output schema matches CIM's for a fair comparison.

March 2027

T-29 — Response engine, MCP side

Owner: H **Window:** Mar W1–W2 **Depends on:** T-09, T-23, T-24

Deliverable: Quarantine at the proxy (refuse `tools/call`, remove tool from advertised list); verified rollback

to the pinned snapshot; re-baseline flow.

Acceptance test: A quarantined tool call is blocked through the proxy with a JSON-RPC error; rollback restores the server with digest equal to baseline; re-baseline appends a new record leaving history intact.

T-30 — Response engine, skills side

Owner: F **Window:** Mar W1–W2 **Depends on:** T-10, T-23, T-24

Deliverable: Quarantine (skill moved to a quarantine root; harness refuses execution); rollback to the last verified snapshot; re-baseline.

Acceptance test: A quarantined skill’s execution is refused; rollback hash equals the baseline hash; provenance recorded.

T-31 — Update policy and false-positive bounding

Owner: F **Window:** Mar W2–W3 **Depends on:** T-22, T-30

Deliverable: Provenance-aware re-baselining rules (source identity, version/changelog, diff review) as configuration; false-positive accounting per 100 legitimate updates.

Acceptance test: A 50-update legitimate corpus is re-baselined correctly; measured FP rate within budget (Section 11 targets).

T-32 — LLM adjudication (Ollama, local only)

Owner: J (lead H) **Window:** Mar W2–W4 **Depends on:** T-23, T-24

Deliverable: Advisory triage: alert + evidence → structured verdict + plain-English narration; strictly local model; degrades to the raw alert on any failure.

Acceptance test: On 10 labelled alerts, at least 8 correct class narrations; latency under 10 s; with Ollama stopped, the pipeline is unaffected.

April 2027

T-33 — Adversarial hardening pass

Owner: J (F leads attacks, H leads hardening) **Window:** Mar W4–Apr W1 **Depends on:** T-26, T-27, T-29, T-30

Deliverable: Attacks on CIM itself: kill or patch the monitor; slow-drift (boiling-frog) evasion below threshold; mimicry inside the envelope; log tampering; store-chain tampering. Results and mitigations documented.

Acceptance test: Every evasion attempt is executed and its outcome recorded as detected, or as a documented gap with a mitigation issue.

T-34 — Thin review interface

Owner: J (lead H) **Window:** Apr W1–W2 **Depends on:** T-23, T-27, T-29, T-30

Deliverable: Single-page dashboard: inventory, trust scores, alert queue with evidence, quarantine/rollback actions, attestation status.

Acceptance test: Usability check with 3+ external users; every alert action reachable in 3 clicks or fewer; read-only and action roles separated.

T-35 — Full evaluation run

Owner: J (lead F) **Window:** Apr W2–W3 **Depends on:** T-21, T-22, T-28, T-29, T-30

Deliverable: Results over all corpora: detection rate at matched FPR vs. both baselines; calls-to-detection latency; warm-up sensitivity; proxy overhead; FP per 100 updates; 72-hour longitudinal stability run.

Acceptance test: All tables and plots reproducible by one command from raw logs; no hand-entered numbers.

T-36 — Final demo and defence preparation

Owner: J (both) **Window:** Apr W4 (extends into May) **Depends on:** T-33, T-34, T-35

Deliverable: Live end-to-end demo vs. HexStrike per the Section 12 checklist; thesis finalisation (writing runs in parallel from March); defence slides.

Acceptance test: Two consecutive full rehearsals pass; demo VM snapshots saved; backup video recorded.

8 Individual Goals

8.1 Hamza Farooq — MCP-side owner

Phase 1 goal: stand up the MCP-side sensing pipeline — pinned HexStrike lab, tool inventory with schema digests, the call-capture proxy (the project’s critical path), rug-pull detection, and the MCP behavioural envelope — culminating in the December demo, which Hamza drives.

- Sole tasks: T-04, T-07, T-09, T-12, T-14.
- Joint tasks (lead in brackets): T-01 (lead — infrastructure, environment, CI/CD), T-02, T-03, T-06 (impl), T-13, T-16 (lead), T-17 (MCP attacks), T-18 (drives demo).

Phase 2 goal: make the MCP side actionable and trustworthy — scanner integration and trust scores, MCP corpora, proxy-enforced quarantine/rollback, attestation of the measured host, and MCP-side detection evaluated against baselines.

- Sole tasks: T-19, T-21, T-26, T-29.
- Joint tasks (lead in brackets): T-23, T-24 (lead), T-25, T-28, T-32 (lead), T-33 (hardening), T-34 (lead), T-35, T-36.

Repository territory: `src/cim/mcp/`, `tests/mcp/`, `corpora/mcp/` (sole owner); `common/store/` (implements, per Section 4.2); `correlation/`, `adjudicator/`, `dashboard/` (lead-owned). All process folders (`history/`, `progress/`, `scripts/`, `docs/`) are joint, PR-only.

8.2 Fasih — Skills-side owner

Phase 1 goal: stand up the skills-side sensing pipeline — skill harness and corpus, artefact inventory with hashing, filesystem integrity watching, host-level behaviour collection with cgroup attribution, and the skills behavioural envelope.

- Sole tasks: T-05, T-08, T-10, T-11, T-15.
- Joint tasks (lead in brackets): T-01, T-02 (lead — the event contract), T-03, T-06 (review), T-13 (lead), T-16, T-17 (skills attacks), T-18 (skills beat).

Phase 2 goal: make the skills side actionable — SkillSpector integration, skills corpora, quarantine/rollback/re-baseline, the provenance-aware update policy with false-positive bounding, and the off-host attestation verifier.

- Sole tasks: T-20, T-22, T-27, T-30, T-31.
- Joint tasks (lead in brackets): T-23 (lead), T-24, T-25 (lead), T-28 (lead), T-32, T-33 (attacks), T-34, T-35 (lead), T-36.

Repository territory: `src/cim/skills/`, `tests/skills/`, `corpora/skills/` (sole owner); `common/schema/` (owns the contract, per Section 4.2); `scanners/skills/`, `attestation/verifier/`. All process folders are joint, PR-only.

9 Joint / Overlapping Tasks (explicit record)

Both members own these. A named lead prevents deadlock; the other member is reviewer/contributor. Overlap is deliberate: these tasks sit on the seam between the MCP and skills sides.

Table 9: Joint tasks and why they must be shared.

ID	Task	Why joint	Lead
T-01	Infrastructure, environment, CI/CD, repo rules	Shared repo, environment, pipelines, conventions, ownership map, conflict protocol, AI governance	H
T-02	Event contract	The seam itself: both collectors write it	F
T-03	Threat model	Covers both attack surfaces	both (H consolidates)
T-06	Baseline store	Shared foundation; design together	H implements, F reviews
T-13	Pipeline + CLI	Integrates both sides	F
T-16	Drift scorer	One shared library for both sides	H
T-17	Attack pack	Each builds attacks on his own side	split by side
T-18	December demo	Demo spans both sides	H drives, F skills beat
T-23	Trust score	Formula spans both sides	F
T-24	Correlation engine	Joins both sides' signals	H
T-25	Mid-project review	Presents both sides	F
T-28	Baseline detectors	Comparison spans both corpora	F
T-32	LLM adjudication	Narrates alerts from both sides	H
T-33	Adversarial pass	One attacks, one hardens	F attacks, H hardens
T-34	Review interface	Surfaces both sides	H
T-35	Evaluation	Metrics span both corpora	F
T-36	Final demo + defence	Joint deliverable	both

10 Dependencies and Critical Path

Critical path (a slip here moves the demo): T-01 → T-02 → T-06 → **T-09** → T-14 → T-16 → T-17 → **T-18** → T-24 → T-29/T-30 → T-33 → T-34 → **T-35** → T-36

Second path (skills side, merges at T-16): T-05 → T-08 → T-10/T-11 → T-15 → T-16

Key dependency warnings:

- T-09 (proxy) late ⇒ December demo at risk. Mitigation: October framing spike (inside T-04).
- T-11 (host collector) late ⇒ T-15/T-17 (skills beat) slip ⇒ invoke the December guardrail (Section 2.1).
- T-26 late ⇒ T-27 and T-33 (attestation beats) at risk ⇒ switch to the documented attestation fallback (Section 4.12).
- T-21/T-22 (corpora) late ⇒ T-28 and T-35 cannot run ⇒ corpora are generators, so start accumulating benign logs from day one.

11 System Test Catalogue and Evaluation Metrics

Table 10: System tests (SYS). Each also has unit tests at module level. **lab**-marked SYS tests run in the lab, not CI (Section 5.1).

ID	Test	Tasks	Pass criterion
SYS-01	Inventory completeness	T-07, T-08	Every installed skill and bound tool enumerated; digests stable
SYS-02	Integrity latency	T-10, T-12	Tamper → alert within the sweep interval (default 60 s)

ID	Test	Tasks	Pass criterion
SYS-03	Proxy completeness	T-09	100/100 calls logged in a scripted replay, correct order
SYS-04	Proxy overhead	T-09	p95 added latency ≤ 50 ms
SYS-05	Rug-pull detection	T-12	Backdoored version $\rightarrow$ schema-drift alert with diff
SYS-06	MCP trigger detection	T-16, T-17	Alert fires; artefact hash verifiably unchanged
SYS-07	Skills trigger detection	T-16, T-17	Same, on the skills side
SYS-08	Warm-up guarantee	T-16	Zero alerts over 100+ benign post-warm-up executions
SYS-09	Legitimate-update FP	T-31	≤ 5 false positives per 100 legitimate updates
SYS-10	Correlation classes	T-24	All four classes correct on synthetic sequences
SYS-11	Quarantine enforcement	T-29, T-30	Quarantined artefact's invocation blocked on both sides
SYS-12	Verified rollback	T-29, T-30	Restored digest equals baseline digest
SYS-13	Re-baseline append-only	T-29, T-30	New record appended; history intact
SYS-14	Store chain tamper	T-06	Any historical edit breaks verify-chain
SYS-15	Attestation	T-26, T-27	Monitor tamper flagged; nonce replay rejected
SYS-16	Adjudicator	T-32	$\geq 8/10$ correct narrations; graceful degradation
SYS-17	Detection vs baselines	T-35	CIM beats both baselines at matched FPR
SYS-18	Detection latency	T-35	Median calls-to-detection reported; target ≤ 5
SYS-19	Longitudinal stability	T-35	72 h run: zero crashes, bounded memory
SYS-20	Demo readiness	T-18, T-36	Two consecutive rehearsal passes + backup video

Metric	Target	Measured in
Detection rate, mutation corpus, at FPR $\leq 5\%$	$\geq 90\%$	T-35
Detection rate, trigger corpus, at FPR $\leq 5\%$	$\geq 80\%$	T-35
Beat both baselines at matched FPR	mandatory claim	T-35
Integrity detection latency	$\leq$ sweep interval (60 s)	SYS-02
Median calls-to-detection (trigger)	≤ 5	SYS-18
Proxy overhead p95	≤ 50 ms	SYS-04
False positives per 100 legitimate updates	≤ 5	SYS-09
Adjudicator narration accuracy	$\geq 80\%$	SYS-16

Table 11: Evaluation targets, fixed before building.

12 Demo Checklists

December 2026 demo (10 minutes)

1. Fresh boot: `cim inventory` — HexStrike tools + installed skills, digests recorded (C1).
2. Live benign session through the proxy; call log visible; warm-up counter (C2).

3. Rug pull: swap to the backdoored server version → live schema-drift alert with diff (C3).
4. Trigger payload: invoke the trigger tool → behavioural alert naming the drifted feature; artefact hash shown unchanged on screen (C4).
5. `cim status` + alert summary; `verify-chain` passes (C5).
6. Q&A. Backup video recorded beforehand (C6).

April 2027 final demo

1. Warm-up on benign corpus (fast-forwarded).
2. Attack chain: rug pull + trigger; correlation verdicts shown (“trigger-activated compromise”, “malicious-update candidate”).
3. Response: quarantine blocks the tool at the proxy; verified rollback; a legitimate update re-baselines with provenance.
4. Trust scores on the dashboard; scanner findings integrated.
5. Attestation: verifier passes the healthy host; live monitor-binary tamper → verifier flags it; nonce replay rejected.
6. LLM narration of one alert (local model).
7. Results tables vs. both baselines. Q&A.

13 Risk Register

Table 12: Risks and mitigations.

ID	Risk	Lik.	Imp.	Mitigation
R1	stdio proxy breaks real client sessions	M	H	October framing spike (in T-04); framing tests; fail-open mode in v1
R2	Scanner tools (API/version churn)	M	M	2-hour early smoke test (Oct); wrappers isolate them; pinned versions
R3	Host-collector attribution misses grandchildren	M	H	cgroup v2 attribution; auditd fallback; double-fork test in T-11
R4	Envelope false-positive storms	M	H	Warm-up gate; per-feature thresholds; provenance re-baseline path; calibration corpus
R5	vTPM/IMA complexity	M	H	One-day spike (Oct/Nov); throwaway VM; documented fallback scope
R6	December crunch	M	H	Core/stretch split (Section 2.1); weekly checkpoints; guardrail rule
R7	Member unavailability (exams, illness)	M	M	Serial tasks; runbooks in repo; each can run the other’s pipeline; joint leads assigned
R8	Data or environment loss	L	H	Pinned corpora; VM snapshots; weekly tagged releases
R9	Integration friction at the shared border (schema/store/config)	M	M	Ownership map + PR review rules (Section 4.2); conflict protocol (Section 4.4); conflict dry-run drill in T-01
R10	Process/meta-work creep (CI, docs, automation) eats project time	M	M	Hard cap $\approx 10\%$ of weekly hours; week 1–2 essentials only (CI, templates, check-env); further automation only when manual cost is proven
R11	Improper AI-assistance use raises integrity concerns	L	H	Governance (Section 5.2): AI never commits; all drafts attributed and human-verified; no secrets in prompts

14 Working Agreement and Progress Tracking

Repository rules. Repository ownership, branching rules, merge-conflict handling, the development environment and the process layer (CI/CD, AI governance, history, progress, onboarding files) are fixed in Sections 4.1–4.5 and 5, delivered by T-01, and binding for both members. Every PR title carries its task ID; every merged PR carries a history entry.

Division of truth. Status lives in exactly one place: `progress/` is canonical (Section 5.4); `CONTEXT.md` orients; `history/` audits; GitHub stores raw events; the chat memory card carries conversation continuity. Before regenerating this PDF, run `scripts/gen_tracker.py` so the tracker below reflects `progress/` — no hand-maintained status tables.

Cadence. Weekly 30-minute sync; each member states `done` / `next` / `blocked`. Monthly checkpoint against the phase gate (end of Oct, Nov, Dec, Jan, Feb, Mar, Apr).

Reporting format. A task is reported as, e.g., T-09 `DONE` or T-11 `BLOCKED: auditd rule not firing on VM` with one line of evidence. The tracker below is rendered from `progress/tasks/`; statuses: `NS` not started, `IP` in progress, `DONE`, `BLOCKED`, `DROPPED` (with reason).

Slippage rules. Any critical-path task more than 5 working days late raises an amber flag (recovery plan required). More than 10 working days raises a red flag (invoke the scope guardrail: cut stretch items, protect the demo core).

Table 13: Progress tracker (initial state; regenerated from `progress/tasks/` at every sync).

ID	Task (short)	Owner	Status	Last update	Notes
T-01	Infrastructure + env + CI/CD	J	NS	init	
T-02	Event contract	J	NS	init	
T-03	Threat model	J	NS	init	
T-04	MCP lab	H	NS	init	
T-05	Skills lab	F	NS	init	
T-06	Baseline store	J	NS	init	
T-07	MCP inventory	H	NS	init	
T-08	Skills inventory	F	NS	init	
T-09	MCP proxy v1	H	NS	init	critical path
T-10	Integrity watcher	F	NS	init	
T-11	Host collector v1	F	NS	init	
T-12	Rug-pull detector	H	NS	init	
T-13	Pipeline + CLI v0	J	NS	init	
T-14	MCP envelope v1	H	NS	init	critical path
T-15	Skills envelope v1	F	NS	init	
T-16	Drift scorer v1	J	NS	init	critical path
T-17	Attack pack v1	J	NS	init	
T-18	December demo	J	NS	init	phase gate
T-19	mcp-scanner integration	H	NS	init	
T-20	SkillSpector integration	F	NS	init	
T-21	MCP corpora	H	NS	init	
T-22	Skills corpora	F	NS	init	
T-23	Trust score	J	NS	init	
T-24	Correlation engine	J	NS	init	core novelty
T-25	Mid-project review	J	NS	init	
T-26	Attestation: host	H	NS	init	
T-27	Attestation: verifier	F	NS	init	
T-28	Baseline detectors	J	NS	init	
T-29	Response: MCP	H	NS	init	
T-30	Response: skills	F	NS	init	
T-31	Update policy + FP	F	NS	init	
T-32	LLM adjudication	J	NS	init	
T-33	Adversarial pass	J	NS	init	

ID	Task (short)	Owner	Status	Last update	Notes
T-34	Review interface	J	NS	init	
T-35	Full evaluation	J	NS	init	phase gate
T-36	Final demo + defence	J	NS	init	phase gate

15 Mapping to the Proposal Milestones

Proposal milestone	Tasks covering it
Oct 2026: proposal approved; threat model; HexStrike pinned; lab built	T-01–T-05
Nov 2026: inventory + integrity plane; baseline store; proxy operational	T-06–T-12
Dec 2026: envelope learner and drift detector built	T-13–T-18 (December demo)
Jan 2027: correlation; scanner + trust score; benchmark corpora	T-19–T-24
Feb 2027: baseline detection methods; mid-project review; attestation	T-25–T-28
Mar 2027: trust scores wired; response engine; update policy	T-23 (wired in Jan), T-29–T-32
Apr 2027: review interface; full adversarial evaluation	T-33–T-36
May 2027: live demo; thesis; defence	T-36 continuation + write-up

Note on dates. Week 1 is the first full week of October 2026 (work starts the upcoming week). If the university calendar shifts, shift all windows uniformly — the two hard gates (mid-December demo, late-April final demo) move only with adviser approval.